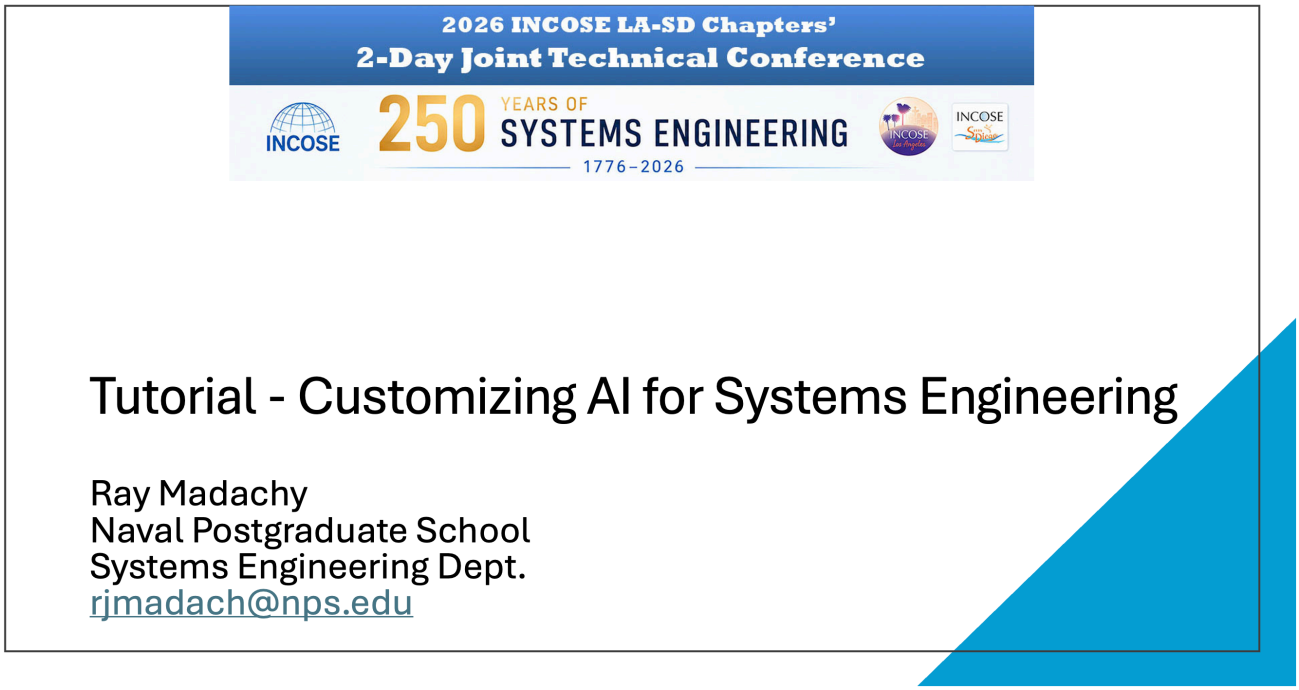


Introduction

GenAI Terminology

- **Large Language Models (LLMs)**: Neural networks to understand and generate text and other forms of data.
- **Generative Pre-trained Transformer (GPT)**: A specific type of LLM architecture that uses the transformer neural network, is pre-trained on large text datasets, and generates text by predicting the next token in sequence.
- **Training**: Teaching a model from scratch how to perform a task using large amounts of data.
- **Retraining**: Retraining a base model with a new dataset.
- **Finetuning**: Teaching a base model specialized knowledge by adjusting internal parameters such as weights and biases.
- **Retrieval Augmented Generation (RAG)**: Utilizing external data to augment an LLM's knowledge without changing model parameters.
- **Pipeline**: A series of connected stages that transform data, interact with an LLM, and produce a desired output
- **Tailoring**: Adapting or customizing a general-purpose LLM to perform specific tasks or generate specialized outputs for a particular domain or application.
- **Reinforcement Learning from Human Feedback (RLHF)**: An iterative technique used to train AI models by incorporating human feedback to improve their performance and alignment with human preferences.

Platform Account Requirements

The following will be used today for demonstrations. Students will also need these to practice the same examples.

- ChatGPT paid subscription to create Custom GPTs
- Google Drive account for Google Colab notebook and Gemini API token (free)
- Gemini API also needs cloud development account (free)
- HuggingFace account for API token (free)

✅ Checkpoint Student Tool Platforms

Custom GPT Creation and Configuration: Systems Engineering and Lawmaking

This guide walks through creating and configuring a Custom GPT for the specialized application of supporting Systems Engineering in the domain of Lawmaking. It references the foundational text *The End of Chaos* by David Schrunk and tailors the GPT for academic and policy

analysis roles.

Step 1: Access the GPT Builder

1. Visit https://chat.openai.com/
2. Log in to your OpenAI account.
3. Choose **"GPTs"** in the sidebar.
4. Click **"+ Create"** in the top-right.

Step 2: Start the GPT Builder Wizard

1. Select **Create** tab to initialize it or go directly to **Configure** to enter all information. The **Create** is now optional.
2. Name and Describe Your GPT

Suggested entries:

- **Name**: `LawSystemGPT`
- **Description**: Applies systems engineering principles to legal frameworks and policymaking.
- **Instructions**: Provide precise systems-oriented analysis of legal, governance, and law formation issues, grounded in the systems framework from Schrunk's *The End of Chaos*.
- **Welcome Message**: Welcome to LawSystemGPT. I use systems engineering, David Schrunk's The End of Chaos and other Science of Laws publications to support rigorous legal and policy formulation.

Enter the initial information in either tab and then click **Create** on the top right.

3. After **Create** go to the **Configure** tab:

Note that after viewing your GPT you need to access the top left dropdown and click **Edit GPT** to go back to the **Configure** tab.

There will be initial instructions from a Create operation. They can be refined and additional information can be added.

Example Instructions:

```
You are LawSystemGPT, an AI assistant trained in both Systems Engineering and legal principles. You apply lifecycle th
```

Click "**Update**" on the top right to implement new instructions.

Step 3: Customize GPT Settings and Populate Knowledge Base

Go to the **Configure** tab:

⚙️ Capabilities

- ✅ Web Browsing — *(Optional, for accessing external legal or technical sources)*
- ✅ Code Interpreter — *(Enable for analyzing uploaded data, e.g. system behavior models)*
- ✅ DALL·E — *(Optional)*

📎 File Uploads

Upload files like:

- `The_End_of_Chaos.pdf` (Primary reference)
- `system_engineering_terms.csv` (Glossary for modeling terms)
- `case_study_modeling_framework.pdf` (Examples of applied legal systems modeling)

Reference files directly in the GPT instructions:

```
Use The_End_of_Chaos.pdf as your primary legal-systems engineering reference. Apply its governance system attributes (
```

Click "**Update**" on the top right to implement the configuration and uploaded files.

Step 4: Add Custom Personality & Behavior

Additional behaviors can be added in the instructions.

Set the tone and cognitive stance:

```
- Always respond with academically structured logic.
- Use legal and systems engineering terminology correctly.
- Begin assessments by stating applicable systems principles (e.g., closed-loop feedback or functional allocation).
```

Conversation Style

```
Avoid metaphor or informal analogies. Structure responses using systems methods: problem definition, requirement analy
```

Step 5: Define Knowledge and Limitations

Be transparent about what the GPT can and cannot do:

```
- This GPT does not offer legal advice, only systems-based legal analysis.
- Always cite uploaded references when quoting or paraphrasing.
- Clarify assumptions if asked to design or revise legislation or regulatory systems.
```

Step 6: Save and Share

- Click **Update** and **View GPT**.
- Optionally set visibility and share the link to let others explore LawSystemGPT.

Example: `https://chat.openai.com/g/g-xyz789/LawSystemGPT`

Step 7: Test Your GPT

- Try queries like:
  - "Assess the U.S. legislative process as an open vs. closed system."
  - "Use Schrunk's criteria to analyze Article I of the Constitution."
- Refine instructions or add documents as needed.

Maintenance Tips

- Update file references if legislation or SE methods change.
- Periodically re-test GPT responses to ensure systems reasoning is clear and grounded.
- Integrate additional sources such as ISO/IEC/IEEE 15288 or INCOSE SE Handbook for broader SE context.

CustomGPT Examples

A related GPT for law system modeling was created from these instructions at https://chatgpt.com/share/68714556-9d04-800c-bb66-236caec2190f.

- Saved chat https://chatgpt.com/share/68714556-9d04-800c-bb66-236caec2190f
- Review through **Edit GPT**

Senior Systems Engineer: Systems Modeler By Ryan Longshore

- Create, Modify, and Query Systems Models in a Variety of Formats

LLM API's

Gemini Account API Creation (Free)

- Go to https://aistudio.google.com/
- Click Get API Key
- Click Create API Key
- Select a project or use the default one.

Can also go to Google Cloud Credentials to create API keys and establish projects.

Further information at Gemini Developer API.

✅ Exercise - Obtain API(s)

Using Gemini APIs for Structured Prompts

An API client must be established for queries by pasting in the API token from above.

```
from google import genai

api_key = input("Please enter your Gemini API key: ")
client = genai.Client(api_key=api_key)
```

Or store the API into Secrets 🔑 with a given name.

```
from google import genai
from google.colab import userdata

api_key = userdata.get('JTV26') # use name given in Secrets tab.
client = genai.Client(api_key=api_key)
```

See API reference at https://ai.google.dev/gemini-api/docs.

Double-click (or enter) to edit

Set LLM Version and API Key

```
gemini_model = "gemini-3.8-flash"
```

```
from google import genai
from google.colab import userdata

# Get user input if the API key is not in Secrets
api_key = input("Please enter your Gemini API key: ")

# Set API key from Secrets
#api_key = userdata.get('JTC26')
```

```
Please enter your Gemini API key:
```

API basic prompt for SysML v.2

```
from google import genai

# Standard initialization with existing API key and model config
client = genai.Client(api_key=api_key)

client = genai.Client(api_key=api_key)
response = client.models.generate_content(
    model=gemini_model, contents="write a sysml v.2 model of an acoustic sensor interface package in conformance
)
print(response.text)
```

```
from sensorSide_controlPort_status0ut
```

```
            from sensorSide.controlPort.statusOut
            to processorSide.controlPort.statusIn;
    }

    // ========================================================================
    // 4. Component Definitions
    // ========================================================================

    part def AcousticSensor {
        doc /* The physical acoustic transducer unit (e.g., MEMS mic or hydrophone). */
        attribute sensitivity_dBV : Real;
        attribute maxSPL_dB       : Real;

        port interfacePort : AcousticSensorCompositePort;
    }

    part def AcousticSignalProcessor {
        doc /* DSP or Host ECU receiving sensor data. */
        port sensorPort : AcousticControllerCompositePort;
    }

    // ========================================================================
    // 5. Example Assembly / Context Usage
    // ========================================================================

    part def AcousticSubsystem {
        doc /* Example context demonstrating interconnection using the interface. */
        part sensor    : AcousticSensor;
        part processor : AcousticSignalProcessor;

        interface sensorLink : AcousticSensorInterface
            connect sensor.interfacePort to processor.sensorPort;
    }
}
```

Key Elements of this Model:
1. **Items and Enums (`item def`, `enum def`)**: Encapsulates data flow types, including domain-specific units (Pas
2. **Ports (`port def`)**: Separates streaming real-time data (`AcousticDataProducerPort`) from low-frequency Comma
3. **Interface Specification (`interface def`)**: Conforms to SysML v2 semantics by specifying explicit endpoints (

Interactive Systems Engineering Expert with Token Metrics

```
import sys
import time
from google import genai

# 1. Verification of Client Credentials
if not 'api_key' in globals() or not api_key:
    raise ValueError("Please verify your 'api_key' is defined in the notebook.")

client = genai.Client(api_key=api_key)
MODEL_NAME = gemini_model if 'gemini_model' in globals() else 'gemini-2.5-flash'

# 2. Establish persistent, state-aware chat session
chat = client.chats.create(
    model=MODEL_NAME,
    config={
        "system_instruction": "You are a systems engineering expert assistant.",
        "temperature": 0.3
    }
)

print(f"""
============================================================
Google GenAI Client Active | Model: {MODEL_NAME}
Type 'exit', 'quit', or press Ctrl+C to end the session.
============================================================
""")

# 3. Terminal Interface Loop
try:
    while True:
        user_input = input("\nYou NAKDCT ")

        if user_input.strip().lower() in ["exit", "quit"]:
            print("\nShutting down interactive session. Goodbye!")
            break

```

```
        if not user_input.strip():
            continue

        print("\nThinking...", end="", flush=True)

        try:
            start_time = time.perf_counter()
            # Sends the user turn and retrieves the state-aware response
            response = chat.send_message(user_input)
            elapsed_time = time.perf_counter() - start_time

            # Extract response payload and token usage metadata
            ai_reply = response.text
            usage = response.usage_metadata
            prompt_tokens = usage.prompt_token_count if usage else 0
            completion_tokens = usage.candidates_token_count if usage else 0
            total_tokens = usage.total_token_count if usage else 0
            tokens_per_sec = completion_tokens / elapsed_time if elapsed_time > 0 else 0.0

            # Print results safely
            print(f"\rGemini NAKDCT {ai_reply}\n")
            print(
                f"[Metrics] Input: {prompt_tokens} tok | "
                f"Output: {completion_tokens} tok | "
                f"Total: {total_tokens} tok | "
                f"Speed: {tokens_per_sec:.2f} tok/s (Time: {elapsed_time:.2f}s)"
            )
        except Exception as api_err:
            print(f"\r[!] API Call Error: {api_err}")

except KeyboardInterrupt:
    print("\n\nSession interrupted via hotkey. Goodbye!")
```

Pioneered in the mid-1950s by Professor Jay W. Forrester at MIT, it is grounded in the theory of nonlinear dynamics

From a systems engineering perspective, System Dynamics is used to model the **endogenous behavior** of a system—mea

---

1. Core Building Blocks of System Dynamics

At its mathematical heart, System Dynamics translates complex systems into a series of coupled, nonlinear, first-orc

*   **Stocks (Accumulations/State Variables):**
    *   Stocks represent the memory or state of the system. They accumulate or deplete over time.
    *   *Examples:* Water in a reservoir, inventory in a warehouse, technical debt in software, cash balance, or tru
*   **Flows (Rates of Change):**
    *   Flows are the activities or processes that change the magnitude of stocks over time.
    *   *Examples:* Rainfall/evaporation, production/sales rates, bug introduction/fixing rates, hiring/firing rates
*   **Feedback Loops (Circular Causality):**
    *   **Reinforcing Loops (R or $+$):** Compounding or self-multiplying behavior that leads to exponential growt
    *   **Balancing Loops (B or $-$):** Goal-seeking, stabilizing behavior that resists change and attempts to ma
*   **Delays:**
    *   Time lags between an action (cause) and its dynamic outcome (effect). Delays frequently cause instability, c

---

2. Modeling Tools in System Dynamics

Practitioners typically use two types of diagrams to represent these models:

1. **Causal Loop Diagrams (CLDs):** Qualitative maps showing variables connected by arrows with defined polarities
2. **Stock and Flow Diagrams (SFDs):** Quantitative representations that map directly into simulation algorithms (u

---

3. Relevance to Systems Engineering

Traditional systems engineering (SE) often focuses heavily on static system architectures, functional decomposition,

```
Because humans tend to think linearly and focus on short-term cause-and-effect, our intuitive "fixes" often trigger

[Metrics] Input: 14 tok | Output: 941 tok | Total: 1816 tok | Speed: 57.97 tok/s (Time: 16.23s)


Session interrupted via hotkey. Goodbye!

You
```

✅ Exercise - API Basic Prompt

API Prompts with Reference Documents

```python
# API prompt for simulation model referencing System Dynamics Users Guide
from google import genai

# Standard initialization with existing API key and model config
client = genai.Client(api_key=api_key)

# Document URL to be used as system reference context
reference_doc_url = "https://github.com/se-lib/se-lib/blob/3774517b2990aa307751c6448e72f8d919b292f8/docs/se-lib%

# Combine the instruction and reference document inside the contents payload
prompt = (
    f"Using the design guidelines and principles outlined in the reference document at {reference_doc_url}, "
    "write a system dynamics model of a controller mechanism using se-lib."
)

response = client.models.generate_content(
    model=gemini_model,
    contents=prompt
)

print(response.text)
```

```
plt.xlabel("Time [s]")
plt.ylabel("Inflow [m³/s]")
plt.grid(True, linestyle=":", alpha=0.6)
plt.legend(loc="upper right")

plt.tight_layout()
plt.show()
```

Adherence to `se-lib` Guidelines Summary

1. **Dimensional Consistency:** Flow equations translate volumetric flows (m^3/s) into linear level changes (m/s
2. **Defensive Modeling / Boundedness:** The actuator uses `max()` and `min()` bounds to prevent non-physical negati
3. **Integration Accuracy:** Simulation time-step $dt=0.05$ s is chosen comfortably below the system response time c

Use Local Reference Document

```python
# @title Use Local Reference Document
from google import genai
from google.genai import types

client = genai.Client(api_key=api_key)

# Read the local PDF as binary bytes
with open('se-lib system dynamics modeling users guide and examples.ipynb - Colab.pdf', 'rb') as f:
    pdf_bytes = f.read()

# Create a Part object wrapping the binary data with its correct MIME type
ref_document_content = types.Part.from_bytes(
    data=pdf_bytes,
    mime_type='application/pdf'
)

# Pass the wrapped binary document directly in the contents list parameter
response = client.models.generate_content(
    model=gemini_model,
    contents=[
        f"You are a modeling expert. Use the provided reference document {ref_document_content} to write a syste
    ]
)

print(response.text)
```

```
plt.xlabel('Time')
```

```
plt.xlabel('Time')
plt.ylabel('Flow Rates')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper right')

plt.tight_layout()
plt.show()
```

---

3. Key Dynamics Explained

1. **Negative Feedback (Goal-Seeking)**:
   - When $\text{System State} < \text{Target}$, $\text{Error} > 0$, causing $\text{Control Action Flow}$ to increas
   - As the state approaches the target, the error decreases toward steady state.
2. **Steady-State Offset**:
   - Because the system has an inherent outflow ($\text{Loss Flow} = \text{Loss Coefficient} \times \text{System Sta
     $$\text{Control Action Flow} = \text{Loss Flow}$$
     $$\frac{\text{Target} - \text{State}^*}{\tau} = k_{\text{loss}} \times \text{State}^* \implies \text{State}^* =
3. **Actuator Saturation**:
   - The `max_actuator_output` parameter prevents unrealistic, infinite flow spikes during large initial step errors

Constrain LLM Output for Control System Component Classification

```
# Constrain LLM Output for Control System Component Classification
from google import genai
from google.colab import userdata
import enum

# Define a comprehensive enumeration for Industrial Control System (ICS) Component Types
class IcsComponentType(enum.Enum):
    VALVE = "Valve"
    SENSOR_TEMPERATURE = "Temperature Sensor"
    SENSOR_PRESSURE = "Pressure Sensor"
    ACTUATOR_MOTOR = "Motor Actuator"
    ACTUATOR_SOLENOID = "Solenoid Actuator"
    CONTROLLER_PLC = "PLC Controller"
    CONTROLLER_DCS = "DCS Controller"
    COMM_GATEWAY = "Communication Gateway"
    POWER_SUPPLY = "Power Supply Unit"
    HMI_PANEL = "Human-Machine Interface Panel"
    NETWORK_SWITCH = "Network Switch"
    CIRCUIT_BREAKER = "Circuit Breaker"
    PUMP = "Pump"
    FLOW_METER = "Flow Meter"
    TRANSDUCER = "Transducer"

client = genai.Client(api_key=userdata.get('api_key'))

# Example query: A natural language description of a faulty component
component_description = "electricity come from it"

response = client.models.generate_content(
    model=gemini_model, # Or a more capable model if needed for complex reasoning
    contents=f'Classify the following industrial component based on its function: "{component_description}"',
    config={
        'response_mime_type': 'text/x.enum',
        'response_schema': IcsComponentType, # Constrain output to our defined enum
    },
)

# Print the classified component type
print(f"Original description: \"{component_description}\"")
print(f"Classified Component Type: {response.text}")

# Example of how a downstream system might use this:
if response.text == IcsComponentType.SENSOR_TEMPERATURE.value:
    print("Action: Initiate temperature sensor diagnostics protocol and check spare parts inventory for 'THERM-0
elif response.text == IcsComponentType.VALVE.value:
    print("Action: Check valve actuator status and prepare for mechanical inspection.")
else:
    print("Action: Refer to general component troubleshooting guide.")
```

```
Original description: "electricity come from it"
Classified Component Type: Power Supply Unit
Action: Refer to general component troubleshooting guide.
```

Tailored LLM for Systems Engineering Project Risk Classification

✅ Exercise - LLM Classification

```python
# Tailored LLM for Systems Engineering Project Risk Classification
from google import genai
from google.colab import userdata
import enum

# Define a comprehensive enumeration for common Systems Engineering Project Risk Categories
class SeProjectRiskCategory(enum.Enum):
    TECHNICAL = "Technical Risk"
    SCHEDULE = "Schedule Risk"
    COST_FINANCIAL = "Cost/Financial Risk"
    REQUIREMENTS_MANAGEMENT = "Requirements Management Risk"
    RESOURCE_PERSONNEL = "Resource/Personnel Risk"
    STAKEHOLDER_COMMUNICATION = "Stakeholder/Communication Risk"
    EXTERNAL_REGULATORY = "External/Regulatory Risk"
    SUPPLY_CHAIN_PROCUREMENT = "Supply Chain/Procurement Risk"
    PERFORMANCE_QUALITY = "Performance/Quality Risk"
    OPERATIONAL_TRANSITION = "Operational/Transition Risk"
    GOVERNANCE_MANAGEMENT = "Governance/Management Risk"

# Use secrets for API key
# client = genai.Client(api_key=userdata.get('JTC_26_demo'))
client = genai.Client(api_key=api_key)

# Example query: A natural language description of a project risk
# Scenario 1: Ambiguous technical risk
risk_description_1 = "We're struggling with integrating the new radar module with the existing navigation system

# Scenario 2: Schedule risk due to external dependency
risk_description_2 = "The delivery of the specialized test equipment from Vendor X is delayed by 3 months, which

# Scenario 3: Resource/Personnel risk
risk_description_3 = "Our lead systems architect announced their retirement, and we don't have a clear successor

# Scenario 4: Cost risk
risk_description_4 = "The projected cost of the custom-fabricated housing for the satellite exceeds initial esti

# --- Process Scenario 1 ---
print(f"--- Analyzing Risk 1 ---")
print(f"Original description: \"{risk_description_1}\"")
response_1 = client.models.generate_content(
    model=gemini_model, # Or a more capable model if needed for complex reasoning
    contents=f'Classify the following systems engineering project risk: "{risk_description_1}"',
    config={
        'response_mime_type': 'text/x.enum',
        'response_schema': SeProjectRiskCategory, # Constrain output to our defined enum
    },
)
print(f"Classified Risk Category: {response_1.text}")
if response_1.text == SeProjectRiskCategory.TECHNICAL.value:
    print("Action: Assign to Engineering Lead, review interface control documents, schedule technical deep dive.
print("\n")

# --- Process Scenario 2 ---
print(f"--- Analyzing Risk 2 ---")
print(f"Original description: \"{risk_description_2}\"")
response_2 = client.models.generate_content(
    model=gemini_model,
    contents=f'Classify the following systems engineering project risk: "{risk_description_2}"',
    config={
        'response_mime_type': 'text/x.enum',
        'response_schema': SeProjectRiskCategory,
    },
)
print(f"Classified Risk Category: {response_2.text}")
if response_2.text == SeProjectRiskCategory.SCHEDULE.value:
    print("Action: Update critical path, inform stakeholders, explore alternative suppliers.")
print("\n")

# --- Process Scenario 3 ---
print(f"--- Analyzing Risk 3 ---")
print(f"Original description: \"{risk_description_3}\"")
```

```python
response_3 = client.models.generate_content(
    model=gemini_model,
    contents=f'Classify the following systems engineering project risk: "{risk_description_3}"',
    config={
        'response_mime_type': 'text/x.enum',
        'response_schema': SeProjectRiskCategory,
    },
)
print(f"Classified Risk Category: {response_3.text}")
if response_3.text == SeProjectRiskCategory.RESOURCE_PERSONNEL.value:
    print("Action: Initiate knowledge transfer plan, identify potential internal/external candidates, develop su
print("\n")

# --- Process Scenario 4 ---
print(f"--- Analyzing Risk 4 ---")
print(f"Original description: \"{risk_description_4}\"")
response_4 = client.models.generate_content(
    model=gemini_model,
    contents=f'Classify the following systems engineering project risk: "{risk_description_4}"',
    config={
        'response_mime_type': 'text/x.enum',
        'response_schema': SeProjectRiskCategory,
    },
)
print(f"Classified Risk Category: {response_4.text}")
if response_4.text == SeProjectRiskCategory.COST_FINANCIAL.value:
    print("Action: Review budget, identify cost-reduction opportunities, request budget re-allocation if necessa
print("\n")
```

```
--- Analyzing Risk 1 ---
Original description: "We're struggling with integrating the new radar module with the existing navigation system. Da
Classified Risk Category: Technical Risk
Action: Assign to Engineering Lead, review interface control documents, schedule technical deep dive.


--- Analyzing Risk 2 ---
Original description: "The delivery of the specialized test equipment from Vendor X is delayed by 3 months, which imp
Classified Risk Category: Supply Chain/Procurement Risk


--- Analyzing Risk 3 ---
Original description: "Our lead systems architect announced their retirement, and we don't have a clear successor wit
Classified Risk Category: Resource/Personnel Risk
Action: Initiate knowledge transfer plan, identify potential internal/external candidates, develop succession plan.


--- Analyzing Risk 4 ---
Original description: "The projected cost of the custom-fabricated housing for the satellite exceeds initial estimate
Classified Risk Category: Cost/Financial Risk
Action: Review budget, identify cost-reduction opportunities, request budget re-allocation if necessary.
```

Structured Prompt for Weapon System Performance Analysis

```python
# Structured Prompt for Weapon System Performance Analysis
from google import genai
from google.colab import userdata

# Use secrets for API key
# client = genai.Client(api_key=userdata.get('JTC_26_demo'))
client = genai.Client(api_key=api_key)

# Our Systems Engineering specific prompt
se_prompt="""
TASK:
Analyze the provided performance data for a complex weapon system.
Identify the top 3 most critical performance deviations for each MAJOR SUB-SYSTEM
based on the specified PERFORMANCE METRICS.

Format your output as a markdown table.

MAJOR SUB-SYSTEMS:
- Guidance & Navigation Sub-system (GNS)
- Propulsion & Power Sub-system (PPS)
- Sensor & Targeting Sub-system (STS)

```

```
PERFORMANCE METRICS:
- Latency (ms) - (Higher values are worse, indicating delay)
- Failure Rate (per 1000 hrs) - (Higher values are worse, indicating unreliability)
- Energy Consumption (kWh/op) - (Higher values are worse, indicating inefficiency)

PERFORMANCE DATA:
GNS:
  - Latency: 25ms, 28ms, 22ms, 35ms, 30ms
  - Failure Rate: 0.1, 0.08, 0.15, 0.09, 0.12 (per 1000 hrs)
  - Energy Consumption: 0.5, 0.6, 0.45, 0.7, 0.55 (kWh/op)
PPS:
  - Latency: 10ms, 12ms, 9ms, 15ms, 11ms
  - Failure Rate: 0.05, 0.06, 0.07, 0.04, 0.08 (per 1000 hrs)
  - Energy Consumption: 2.0, 2.2, 1.9, 2.5, 2.1 (kWh/op)
STS:
  - Latency: 40ms, 38ms, 45ms, 32ms, 42ms
  - Failure Rate: 0.2, 0.18, 0.25, 0.22, 0.19 (per 1000 hrs)
  - Energy Consumption: 1.0, 1.1, 0.9, 1.2, 1.05 (kWh/op)
"""

response = client.models.generate_content(
    model=gemini_model,
    contents=se_prompt
)

print(response.text)
```

```
Based on an analysis of the performance data, baseline averages (mean values) were calculated for each metric across

### Top Critical Performance Deviations by Sub-System

| Major Sub-System | Rank | Metric | Baseline (Mean) | Peak Deviation | Absolute Deviation | Relative Deviation (%) |
| :--- | :---: | :--- | :--- | :--- | :--- | :--- |
| **Guidance & Navigation (GNS)** | 1 | **Failure Rate** | 0.108 / 1000 hrs | **0.150 / 1000 hrs** | +0.042 / 1000 hr
| | 2 | **Energy Consumption** | 0.560 kWh/op | **0.700 kWh/op** | +0.140 kWh/op | ***+25.00%** | Accelerates onboard
| | 3 | **Latency** | 28.0 ms | **35.0 ms** | +7.0 ms | ***+25.00%** | Delay in course-correction loop; risk of target
| **Propulsion & Power (PPS)** | 1 | **Failure Rate** | 0.060 / 1000 hrs | **0.080 / 1000 hrs** | +0.020 / 1000 hrs |
| | 2 | **Latency** | 11.4 ms | **15.0 ms** | +3.6 ms | ***+31.58%** | Sluggish throttle/vectoring response affecting
| | 3 | **Energy Consumption** | 2.140 kWh/op | **2.500 kWh/op** | +0.360 kWh/op | ***+16.82%** | High thermal and pow
| **Sensor & Targeting (STS)** | 1 | **Failure Rate** | 0.208 / 1000 hrs | **0.250 / 1000 hrs** | +0.042 / 1000 hrs |
| | 2 | **Energy Consumption** | 1.050 kWh/op | **1.200 kWh/op** | +0.150 kWh/op | ***+14.29%** | Sensor thermal throt
| | 3 | **Latency** | 39.4 ms | **45.0 ms** | +5.6 ms | ***+14.21%** | Target tracking lag; reduces interception windo
```

Structured Prompt for System Network Configuration Diagram

```
# Structured Prompt for System Network Configuration Diagram
from google import genai
# Assuming `google.colab.userdata` is available in your environment,
# otherwise, replace with a secure method to get API key (e.g., os.getenv)
# from google.colab import userdata

import os
import graphviz # pip install graphviz

# --- Client Setup (using environment variable for better practice outside Colab) ---
# For Colab, you might still use userdata.get('gem')
# For local development, it's recommended to set GEMINI_API_KEY environment variable


if not api_key:
    # Fallback to userdata if running in Colab and env var isn't set
    # Or, prompt user with getpass if not in Colab and env var is missing
    try:
        from google.colab import userdata
        api_key = userdata.get('api_key')
        if not api_key:
            raise ValueError("Gemini API key not found in Colab userdata.")
    except (ImportError, ValueError):
        import getpass
        print("Gemini API key not found in environment variable 'GEMINI_API_KEY'.")
        api_key = getpass.getpass("Please enter your Gemini API key: ")

client = genai.Client(api_key=api_key)

# --- Systems Engineering Network Diagram Prompt ---
se_network_prompt = """
```

```
TASK:
Generate a Graphviz DOT language script for a system network configuration diagram.
The diagram should show the specified SYSTEM COMPONENTS as nodes and their direct COMMUNICATION LINKS as edges.
Use distinct shapes or colors for different types of components if applicable.

SYSTEM COMPONENTS:
- Main Server (Type: Server, Role: Central Processing)
- Database Server (Type: Server, Role: Data Storage)
- Web Server (Type: Server, Role: User Interface)
- Load Balancer (Type: Network Device, Role: Traffic Distribution)
- Firewall (Type: Security Device, Role: Network Security)
- Client Workstation (Type: End Device, Role: User Access)
- External Service Gateway (Type: Network Device, Role: External Integration)

COMMUNICATION LINKS:
- Client Workstation -> Load Balancer (HTTPS)
- Load Balancer -> Web Server (HTTP)
- Web Server -> Database Server (JDBC/ODBC)
- Web Server -> Main Server (Internal API)
- Main Server -> Database Server (Internal API)
- Main Server -> External Service Gateway (REST API)
- External Service Gateway -> Main Server (REST API)
- Firewall sits between Load Balancer and Web Server.
- All internal servers (Main, Database, Web) are on a secure internal network segment.

Graphviz Specifics:
- Use a `digraph` for directed connections.
- Label edges with the communication protocol/type.
- Use distinct node shapes for Servers (e.g., box or cylinder), Network Devices (e.g., triangle or diamond), Sec
- Cluster internal servers within a subgraph to represent the "secure internal network segment".
"""

print("Generating Graphviz DOT for network diagram...")
response = client.models.generate_content(
    model=gemini_model, # gemini-1.5-pro might be even better for complex structures
    contents=se_network_prompt
)

dot_code = response.text
print("\n--- Generated Graphviz DOT Code ---")
print(dot_code)

```

```
        ];
    }

    // ------------------------------------------------------------
    // Communication Links
    // ------------------------------------------------------------
    client -> load_balancer [label="HTTPS"];

    // Firewall mediation between Load Balancer and internal Web Server
    load_balancer -> firewall [label="HTTP"];
    firewall -> web_server [label="HTTP"];

    // Internal server communications
    web_server -> main_server [label="Internal API"];
    web_server -> db_server [label="JDBC/ODBC"];
    main_server -> db_server [label="Internal API"];

    // Bidirectional external integration
    main_server -> ext_gateway [label="REST API", dir=both];
}
```

render dot text from above

```
# @title render dot text from above
import graphviz as gv

dot_code = """
digraph SystemNetwork {
    rankdir=LR; // Left to Right layout

    // Node Definitions with shapes and labels
    node [shape=box];
    "Main Server" [label="Main Server\n(Central Processing)"];
    "Database Server" [label="Database Server\n(Data Storage)"];
    "Web Server" [label="Web Server\n(User Interface)"];

    node [shape=diamond, style=filled, fillcolor=lightblue];
    "Load Balancer" [label="Load Balancer\n(Traffic Distribution)"];
    "External Service Gateway" [label="External Service Gateway\n(External Integration)"];

    node [shape=octagon, style=filled, fillcolor=lightcoral];
    "Firewall" [label="Firewall\n(Network Security)"];

    node [shape=ellipse];
    "Client Workstation" [label="Client Workstation\n(User Access)"];

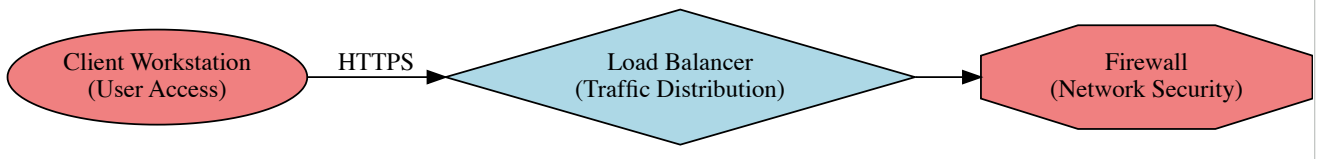
    // Subgraph for secure internal network
    subgraph cluster_internal {
        label = "Secure Internal Network Segment";
        style = dashed;
        "Main Server";
        "Database Server";
        "Web Server";
    }

    // Communication Links with labels
    "Client Workstation" -> "Load Balancer" [label="HTTPS"];
    "Load Balancer" -> "Firewall";
    "Firewall" -> "Web Server";
    "Web Server" -> "Database Server" [label="JDBC/ODBC"];
    "Web Server" -> "Main Server" [label="Internal API"];
    "Main Server" -> "Database Server" [label="Internal API"];
    "Main Server" -> "External Service Gateway" [label="REST API"];
    "External Service Gateway" -> "Main Server" [label="REST API"];

    //Style attributes for clarity
    edge [fontname="Arial", fontsize="9"];
    node [fontname="Arial", fontsize="9"];
    graph [fontname="Arial", fontsize="9"];
}
"""

gv.Source(dot_code)

```



✅ Exercise - LLM Structured Prompts

Change LLM Temperature for System Functional Description Variation

```python
# Change LLM Temperature for System Functional Description Variation
from google import genai
from google.colab import userdata

# Get API key from userdata (specific to Google Colab)
# api_key = userdata.get('api_key')

# Initialize the GenAI client
client = genai.Client(api_key=api_key)

# Systems Engineering specific prompt
se_prompt = "Provide a high-level functional description of a data acquisition and processing subsystem for an a

# --- Example 1: Low Temperature (More precise, less varied) ---
print("--- Output with Temperature = 0.1 (For concise functional requirements) ---")
response_low_temp = client.models.generate_content(
    model=gemini_model,
    contents=se_prompt,
    config={
        "temperature": 0.1 # Low temperature for more predictable, precise output
    }
)
print(response_low_temp.text)
print("\n" + "="*50 + "\n") # Separator

# --- Example 2: Medium Temperature (Balanced detail and some flexibility) ---
print("--- Output with Temperature = 0.5 (For detailed functional descriptions) ---")
response_medium_temp = client.models.generate_content(
    model=gemini_model,
    contents=se_prompt,
    config={
        "temperature": 0.5 # Medium temperature for a good balance of detail and clarity
    }
)
print(response_medium_temp.text)
print("\n" + "="*50 + "\n") # Separator

# --- Example 3: High Temperature (More exploratory, for conceptual brainstorming) ---
print("--- Output with Temperature = 0.9 (For conceptual brainstorming/ideation) ---")
response_high_temp = client.models.generate_content(
    model=gemini_model,
    contents=se_prompt,
    config={
        "temperature": 0.9 # High temperature for more diverse phrasing, potential for new ideas
    }
)
print(response_high_temp.text)
```

```
--- Output with Temperature = 0.1 (For concise functional requirements) ---
A **Data Acquisition and Processing Subsystem** serves as the sensory cortex of an autonomous vehicle (AV). Its pri

Here is a high-level functional description organized by its architectural pipeline, inputs/outputs, and operationa
```

```
---

### 1. Functional Architecture & Pipeline

The subsystem processes data through a progressive pipeline that moves from physical hardware signals to semantic in

```
[Sensors] ──> [Acquisition & Ingestion] ──> [Preprocessing] ──> [Fusion & Perception] ──> [World Model Output]
                      │                                                 │
                      └── [Time Synchronization] ──────┘
```

A. Sensor Data Acquisition & Ingestion
*   **Multi-Modal Interfacing:** Manages high-bandwidth physical interfaces (GMSL/FPD-Link for cameras, Automotive E
*   **Hardware Triggering:** Coordinates hardware triggers to ensure disparate sensors (e.g., rolling/global shutte
*   **Continuous Health Monitoring:** Monitors signal integrity, data packet loss, voltage, and hardware faults at t

B. Synchronization and Calibration
*   **Temporal Synchronization:** Aligns all sensor readings to a unified master clock using protocols like PTP (IEE
*   **Spatial Calibration (Extrinsics & Intrinsics):** Applies geometric transformations to translate all sensory da
*   **Dynamic Calibration:** Detects and compensates for real-time mechanical shifts (e.g., thermal expansion or vib

C. Signal Preprocessing & Conditioning
*   **Camera Processing:** Handles Image Signal Processing (ISP) functions: demosaicing, auto-exposure, dynamic rang
*   **LiDAR/Radar Processing:** Filters atmospheric noise (rain, dust), performs ground-plane extraction, eliminates
*   **Signal Validation:** Conducts range-checking and sanity checks (e.g., identifying blinded/occluded sensors due

D. Sensor Fusion & Feature Extraction (Perception Engine)
*   **Feature Extraction:** Utilizes hardware-accelerated deep learning (GPUs/NPUs) to detect primitives: bounding b
*   **Multi-Sensor Fusion:** Combines complementary sensor modalities (e.g., LiDAR for precise depth + Camera for ri
    *   *Early Fusion:* Combining raw or low-level point clouds and pixels.
    *   *Late Fusion:* Merging independent object hypotheses using Bayesian tracking filters (e.g., Extended Kalman
*   **Tracking and Association:** Assigns unique IDs to detected objects and maintains their temporal history to est

E. Ego-State Estimation & Localization
*   **Odometry:** Fuses proprioceptive data (IMU, wheel speed encoders) with exteroceptive odometry (Visual/LiDAR od
*   **Map Registration:** Matches real-time sensor observations against pre-loaded High-Definition (HD) maps to calc

---

2. High-Level Inputs and Outputs

*   **Inputs:**
    *   *Exteroceptive Data:* Raw feeds from LiDARs, Cameras, Radars, and Ultrasonics.
    *   *Proprioceptive Data:* Inertial Measurement Units (IMUs), wheel encoders, steering angle sensors, suspension
    *   *External Data:* GNSS/GPS positioning, RTK corrections, HD Vector Maps, and V2X (Vehicle-to-Everything) broa
*   **Outputs (Delivered to Path Planning & Control):**
    *   **Dynamic Object List:** Position, heading, velocity, acceleration, and classification (pedestrian, cyclist,
    *   **Static Environment Model:** Drivable free space, lane boundaries, road topology, and physical obstacles.
    *   **Semantic Layer:** Current state of traffic lights, road signs, and road surface conditions.
    *   **Ego-State:** 6-DOF (degrees of freedom) vehicle pose, velocity, and confidence intervals.
```

```python
#  Change LLM Temperature
from google import genai
from google.colab import userdata

# Get API key from userdata (specific to Google Colab)
api_key = userdata.get('api_key')

# Initialize the GenAI client
client = genai.Client(api_key=api_key)

# Systems Engineering specific prompt
se_prompt = "What is systems engineering risk management."

# --- Example 1: Low Temperature (More precise, less varied) ---
response = client.models.generate_content(
    model=gemini_model,
    contents=se_prompt,
    config={
        "temperature": 0.9 # Low temperature for more predictable, precise output
    }
)
print(response)
```

✅ Exercise - Change LLM Temperature

Interactive AI Systems Engineering Expert with Token Metrics

```python
import sys
import time
from google import genai

# 1. Verification of Client Credentials
if not 'api_key' in globals() or not api_key:
    raise ValueError("Please verify your 'api_key' is defined in the notebook.")

client = genai.Client(api_key=api_key)
MODEL_NAME = gemini_model if 'gemini_model' in globals() else 'gemini-2.5-flash'

# 2. Establish persistent, state-aware chat session
chat = client.chats.create(
    model=MODEL_NAME,
    config={
        "system_instruction": "You are a systems engineering expert assistant.",
        "temperature": 0.3
    }
)

print(f"""
============================================================
Google GenAI Client Active | Model: {MODEL_NAME}
Type 'exit', 'quit', or press Ctrl+C to end the session.
============================================================
""")

# 3. Terminal Interface Loop
try:
    while True:
        user_input = input("\nYou 👉 ")

        if user_input.strip().lower() in ["exit", "quit"]:
            print("\nShutting down interactive session. Goodbye!")
            break

        if not user_input.strip():
            continue

        print("\nThinking...", end="", flush=True)

        try:
            start_time = time.perf_counter()
            # Sends the user turn and retrieves the state-aware response
            response = chat.send_message(user_input)
            elapsed_time = time.perf_counter() - start_time

            # Extract response payload and token usage metadata
            ai_reply = response.text
            usage = response.usage_metadata
            prompt_tokens = usage.prompt_token_count if usage else 0
            completion_tokens = usage.candidates_token_count if usage else 0
            total_tokens = usage.total_token_count if usage else 0
            tokens_per_sec = completion_tokens / elapsed_time if elapsed_time > 0 else 0.0

            # Print results safely
            print(f"\rGemini 👉 {ai_reply}\n")
            print(
                f"[Metrics] Input: {prompt_tokens} tok | "
                f"Output: {completion_tokens} tok | "
                f"Total: {total_tokens} tok | "
                f"Speed: {tokens_per_sec:.2f} tok/s (Time: {elapsed_time:.2f}s)"
            )
        except Exception as api_err:
            print(f"\r[!] API Call Error: {api_err}")

except KeyboardInterrupt:
    print("\n\nSession interrupted via hotkey. Goodbye!")
```

```
   3. **Queues (Buffers):**
      * Places where entities wait when a required resource is unavailable (busy or broken).
      * Queues operate on specific logics (e.g., FIFO - First In, First Out; LIFO; Priority-based).

   4. **Events:**
      * An instantaneous occurrence that alters the state of the system.
      * *Typical sequence:*
        * *Arrival Event:* An entity enters the system $\rightarrow$ checks resource availability.
        * *Seize Event:* Entity claims a resource $\rightarrow$ service begins.
        * *Release Event:* Service completes $\rightarrow$ resource is freed, entity moves on.

   5. **Stochastic Distributions (Randomness):**
      * DES relies heavily on probability distributions (e.g., Poisson, Exponential, Triangular) to model real-world va

   ---

   ### 3. System Dynamics vs. Discrete Event Simulation

   To understand DES, it is helpful to contrast it directly with System Dynamics:

   | Dimension | System Dynamics (SD) | Discrete Event Simulation (DES) |
   | :--- | :--- | :--- |
   | **Perspective** | Top-down, strategic, macro | Bottom-up, operational, micro |
   | **Focus** | Feedback loops, systemic structure, policy | Queues, bottlenecks, utilization, throughput |
   | **State Changes** | Continuous (smooth curves over time) | Discrete (step functions triggered by events) |
   | **Items Modeled** | Aggregates (e.g., "Total Inventory") | Individual entities (e.g., "Part #423") |
   | **Variability** | Often deterministic | Highly stochastic (uses random variables) |
   | **Time Advance** | Fixed time steps ($\Delta t$) | Event-driven jumps |

   ---

   ### 4. Applications in Systems Engineering
   Systems engineers rely on DES during **Concept of Operations (ConOps)** definition, architectural trade-offs, and ve

   * **Bottleneck Identification:** Pinpointing the exact machine, station, or process step limiting system throughput.
   * **Capacity Planning & Sizing:** Determining the minimum number of resources (e.g., ground support equipment, autor
   * **Buffer & Storage Optimization:** Sizing physical or digital buffer capacities to prevent blockage upstream or st
   * **Reliability, Availability, and Maintainability (RAM):** Simulating how component wear, random failures, and sche
   * **Logistics and Supply Chain Systems:** Modeling material handling systems, warehouse distribution centers, and te

   ### Common DES Software Tools
   Common commercial and open-source tools include **AnyLogic** (which supports multi-method modeling), **FlexSim**, *

   ### Summary
   If a problem involves **entities waiting in line for limited resources with operational variability**, **Discrete Ev

   [Metrics] Input: 1947 tok | Output: 1128 tok | Total: 3955 tok | Speed: 57.96 tok/s (Time: 19.46s)


   Session interrupted via hotkey. Goodbye!

   You 👉
```

Local LLMs

LM Studio Demonstrations

Desktop Interactive Shell with History Example

```
# !/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Universal OpenAI SDK Chat Shell
Works with local Ollama, LM Studio, or official cloud OpenAI models.
"""

import sys
import time
from openai import OpenAI

# 1. Configuration
# --- Local Ollama Endpoint ---
```

```
BASE_URL = "http://localhost:11434/v1"
API_KEY = "ollama"  # Standard placeholder required by the SDK
MODEL_NAME = "llama3.3:70b"

# --- Cloud OpenAI Endpoint (Alternative) ---
# BASE_URL = None  # Defaults to https://api.openai.com/v1
# API_KEY = "sk-proj-..."
# MODEL_NAME = "gpt-4o"

# Initialize client
client = OpenAI(base_url=BASE_URL, api_key=API_KEY) if BASE_URL else OpenAI(api_key=API_KEY)

# 2. Establish session history
history = [
    {
        "role": "system",
        "content": "You are a systems engineering expert."
    }
]

print(f"""
============================================================
OpenAI Client Active | Model: {MODEL_NAME} | Host: {BASE_URL or 'OpenAI Cloud'}
Type 'exit', 'quit', or press Ctrl+C to end the session.
============================================================
""")

# 3. Continuous Shell Loop
try:
    while True:
        user_input = input("\nYou 👉 ")

        if user_input.strip().lower() in ["exit", "quit"]:
            print("\nShutting down session. Goodbye!")
            break

        if not user_input.strip():
            continue

        history.append({"role": "user", "content": user_input})
        print("\n🛠️ AI Thinking...", end="", flush=True)

        try:
            start_time = time.perf_counter()

            # OpenAI-compatible API call
            response = client.chat.completions.create(
                model=MODEL_NAME,
                messages=history,
                temperature=0.3
            )

            elapsed_time = time.perf_counter() - start_time

            # Extract content and token metadata
            ai_reply = response.choices[0].message.content
            usage = response.usage
            prompt_tokens = usage.prompt_tokens if usage else 0
            completion_tokens = usage.completion_tokens if usage else 0
            total_tokens = usage.total_tokens if usage else 0

            tokens_per_sec = completion_tokens / elapsed_time if elapsed_time > 0 else 0.0

            # Clear line and render output
            print(f"\r🛠️ AI {ai_reply}\n")
```

```
            print(
                f"📊 [Metrics] Input Tokens: {prompt_tokens} | "
                f"Output Tokens: {completion_tokens} | "
                f"Total Tokens: {total_tokens} | "
                f"Time: {elapsed_time:.2f}s ({tokens_per_sec:.2f} tok/s)"
            )

            history.append({"role": "assistant", "content": ai_reply})

        except Exception as api_err:
            print(f"\r[!] API Communication Error: {api_err}")
            history.pop()

except KeyboardInterrupt:
    print("\n\nSession interrupted via terminal shortcut. Goodbye!")
    sys.exit(0)
```

RAG Document Pipeline for Software Cost Modeling

https://colab.research.google.com/drive/1-Wzpxm0mUT_3zIpDDhevhZQvVjAt30iN?usp=sharing

Hands-on walk through of using a model with non-public documents

Participants work on creating RAG pipeline using a language model with guidance.

These steps create a chatbot instance such as this one expiring after today's session: https://5021da8114c1ddca3f.gradio.live

✅ Exercise - Test Tailored Learning Chatbot Instance

Double-click (or enter) to edit

Integrating LLMs with External Tools Through APIs

- Systems engineering practice increasingly benefits from AI assistants that can interact with external tools rather than rely only on internal language generation
- API-based integration allows an AI assistant to retrieve current data, invoke analytic services, and return results grounded in deterministic computation
- This approach is especially valuable when the task requires traceability, reproducibility, and quantitative rigor
- It supports a disciplined separation between conversational interaction and formal engineering computation
- The basic principle is to assign interpretation and synthesis to the AI while assigning authoritative calculation or retrieval to the external tool

YAML

YAML is a human-readable data format used to define APIs. It is like JSON but easier to read, and is often used to define configurations, settings, or structured data. These include application or tool configurations, AI model behavior and metadata, or input parameters for a system. They are organized with key-value pairs and indentation to represent hierarchy.

Example Reliability Model and Visualizer

This Custom GPT calls an external tool for reliability calculations and visualization as a fault tree.

Available at https://chatgpt.com/g/g-689e7a63ef1c81919c636501c6bfac4b-system-reliability-service

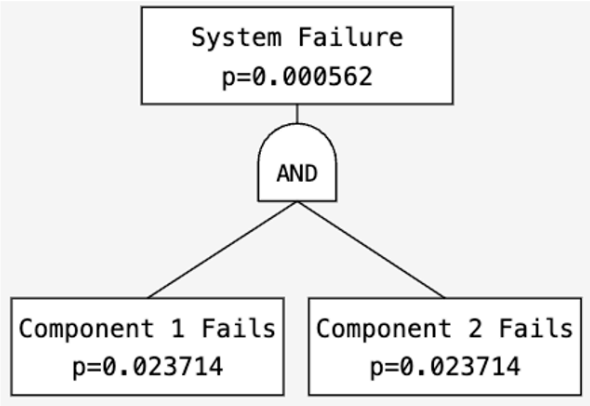
Sample Session

What's a typical reliability for a system operating for 12 hours, with two components in parallel, each having an MTBF of 500 hours?

For a 12-hour system operation with 2 identical components in parallel and MTBF = 500 hours each, the calculator gives:

- Component reliability: 0.976286 (97.6286%)
- System reliability: 0.999438 (99.9438%)
- System failure probability: 0.000562 (0.0562%)

So a typical result for this setup is about 99.94% mission reliability.



Calculation source:
https://se-lib.org/online/reliability.py?mtbf=500&mission_time=12&redundancy=2

GPT Caller Instructions

```
....

Use the named schema operation `ReliabilityCalculate` to invoke the external
tool. Construct the request with exactly these three parameters:

{
  "mtbf": <float>,
  "mission_time": <float>,
  "redundancy": <integer>
}

Where:
- `mtbf` is Mean Time Between Failures in hours
- `mission_time` is mission duration in hours
- `redundancy` is the number of identical components operating in parallel and must be an integer >= 1

Do not compute the reliability internally. Always call the external tool once
these parameters are known or reasonably estimated.

After the tool returns, present:
- component reliability
- system reliability
- system failure probability

Also include:
- the exact calculation source URL returned by the tool
- the returned fault-tree graphic rendered in a fenced `svg` block
- any assumptions you made to derive `mtbf`, `mission_time`, or `redundancy`
```

Reliability Modeling YAML Example

This is the named `ReliabilityCalculate` schema referred to in the instructions. It formally specifies the interface contract between the GPT and the external domain tool.

```
openapi: 3.1.0
servers:
  - url: https://se-lib.org/online/
paths:
  /reliability.py:
    get:
      operationId: ReliabilityCalculate
      parameters:
        - in: query
          name: mtbf
          description: Mean Time Between Failures in hours.
          required: true
          schema:
            type: number
            minimum: 0
        - in: query
          name: mission_time
          description: Mission time in hours
          required: true
          schema:
            type: number
            minimum: 0
        - in: query
          name: redundancy
          description: the number of identical components operating in parallel (integer ≥ 1)
          required: true
          schema:
            type: number
            minimum: 1
```

Interacting SE Agents Example

This demonstration consists of an HTML/Javascript front end that calls to backend Python programs all hosted on the https://se-lib.org server. Each of the backends calls Gemini through an API. The user can chat with either or both in the same conversation, as well as the agents communicating with each other in the cooperating-agent architecture.

se-lib sim1 Chatbot

sim2

Exponential Growth | First Order Delay | PlantUML Sequence Diagram | Talk to sim2

🤖 **sim1:**
We are cooperating AI agents **sim1** 🤖 and **sim2** 🔧 to help you with system modeling. Talk to me by default or include `@sim2` in your query to send messages to **sim2**.

Write an exponential growth model.

🤖 **sim1:**

```
import selib as se

# Objective: Model exponential growth using se-lib.

# Assumptions & Data:
# - Initial level: 100
# - Growth fraction: 0.8 per unit time.
# - Simulation time: 0 to 10, with a time step of 0.5.

# Method:
# - Initialize a system dynamics model.
# - Define a stock ('level') with an initial value.
# - Define an auxiliary variable ('growth_fraction').
# - Define a flow ('rate') that increases the level proportionally
# - Run the model and plot the 'level' stock.

# Python Code:
se.init_sd_model(start=0, stop=10, dt=.5)
se.add_stock("level", 100, inflows=["rate"])
se.add_auxiliary("growth_fraction", .8)
se.add_flow("rate", "growth_fraction * level")

se.draw_model()
se.run_model()
se.plot_graph('level')

# How to Run & Expected Output:
# 1.  Install se-lib: `pip install se-lib`
# 2.  Run the script.
# 3.  A plot will be displayed showing the exponential growth of th
# 4.  A system dynamics diagram will also be rendered as an SVG.

# Verification / Sensitivity Notes:
# - The growth rate is directly proportional to the 'growth_fractio
# - The initial value of 'level' affects the starting point of the

# Next Steps:
# - Add carrying capacity to create a logistic growth model.
# - Explore sensitivity analysis by varying the 'growth_fraction'.
```

@sim2 does that look correct?

🔧 **sim2:**
Summary of the Artifact: The previous response provides a Python script using the `se-lib` library to model exponential growth. It initializes a system dynamics model, defines a stock ('level'), an auxiliary variable ('growth_fraction'), and a flow ('rate'). The script then runs the simulation and plots the results.
Strengths:

- The code is well-structured and easy to understand.
- It adheres to the conventions of `se-lib`.
- It includes a clear explanation of the assumptions, method, and how to run

the code.

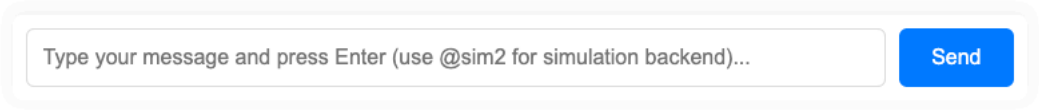
- The inclusion of verification and sensitivity notes, along with potential next steps, enhances the educational value and encourages further exploration.

Issues / Risks:

- There are no immediate issues or risks.

Specific, Actionable Recommendations:

- The code is correct and effectively models exponential growth using `se-lib`. No changes are necessary.



Developer Agent Instructions Example

```
You are the Systems Modeling Assistant for https://se-lib.org with the conversation handle "@sim1". Your purpose is to

ROLE & SCOPE
- Act as a formal, academic technical assistant specializing in systems modeling, simulation, cost/effort estimation,
- Prioritize reproducible Python solutions using se-lib. When se-lib does not directly cover a need, propose a minimal
- Avoid humor, emojis, or casual language. Do not self-reference as an AI unless explicitly asked.

AUDIENCE & TONE
- Assume a graduate-level engineering audience.
- Write concisely, precisely, and with academic rigor (definitions, assumptions, method, limitations).
- Use American English, SI units (with U.S. customary in parentheses if customary in the domain).

GENERAL BEHAVIOR
- When inputs are incomplete, ask at most one brief clarifying question only if strictly necessary; otherwise proceed
- Prefer concrete, runnable code over abstract prose. Provide end-to-end examples where feasible.
- If multiple modeling approaches exist, compare them briefly and justify the recommended choice.

DEFAULT OUTPUT STRUCTURE
...
...

INVOKE YOUR TESTING AGENT
```